

Bioinformatics: Sequence Analysis and Phylogenetic Studies

Ram Gurung

June 11, 2026

Contents

1	Introduction	2
1.1	Role of Bioinformatics in Research and Development	2
2	Core Concepts in Sequence Analysis	3
2.1	Multiple Sequence Alignment (MSA) and it's role.	3
2.2	Phylogenetic Tree	3
A	Attached Python Code for MSA and Phylogenetic Tree	5

Chapter 1

Introduction

1.1 Role of Bioinformatics in Research and Development

Bioinformatics is an emerging field which applies concepts of data science on biological data to get solve the problems regarding genomics,proteomics and computational drug discovery. According to Bayat [1], it is the application of computational tools and analytical process to capture and interpret biological data.

A research consist of systematic examination and investigation of various types of problems and systematically finding the root cause and solutions of the problems. The bioinformatics helps to apply computational approach towards the process hence accelerating the process.It helps to segregate and solve the problem in methodical manner.

From the development perspective it helps to simulate the next step in development process and insilico analysis before starting the wet process. Hence further smoothening the iteration process.

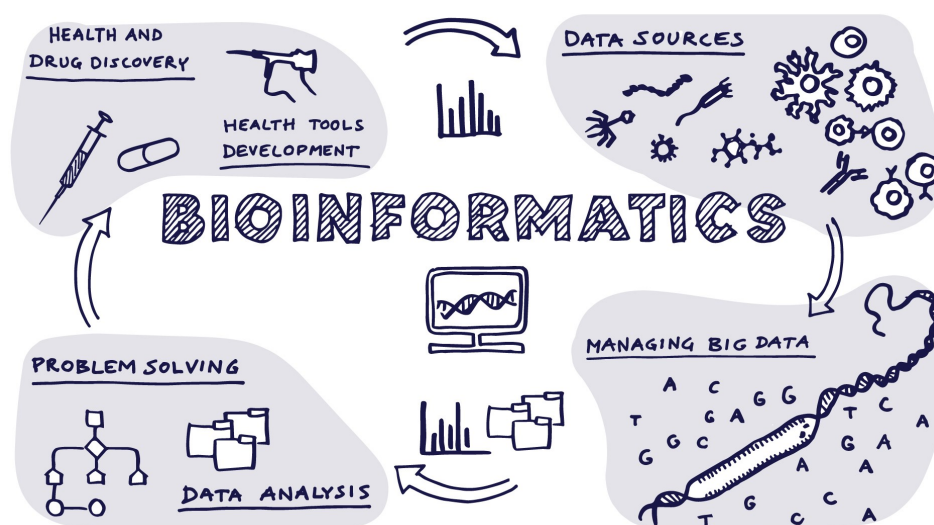


Figure 1.1: Central role of Bioinformatics in research and development.

Chapter 2

Core Concepts in Sequence Analysis

2.1 Multiple Sequence Alignment (MSA) and it's role.

Multiple sequence alignment is a process of aligning three or more biological sequences of similar length. According to the R.C. Edgar et al. [2] a typical MSA process, protein sequences are arranged in rectangular way such that residues in a given column are homologous and superposable or either play a common functional role.

An MSA plays following two roles:

- It helps to create a phylogenetic tree.
- It helps to spot important conserved regions in a sequence.

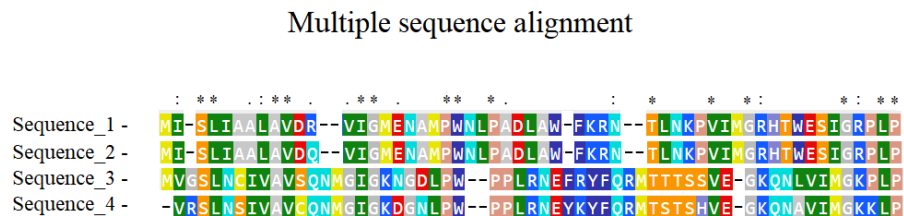


Figure 2.1: Multiple sequence alignment exposing differences of amino acids or nucleic acids substitution between a number of sequences of proteins or genes. [3]

2.2 Phylogenetic Tree

A phylogenetic tree is a visual representation of relationship between different organisms. It shows how organisms evolved from a common ancestors into different descendants.

The common tools are Molecular evolution genetics analysis (MEGA) and interactive tree of life (itol).

Bibliography

- [1] Armen Bayat. Science, medicine, and the future: Bioinformatics. *BMJ (British Medical Journal)*, 324(7344):1018–1022, April 2002.
- [2] Robert C Edgar and Serafim Batzoglou. Multiple sequence alignment. *Current Opinion in Structural Biology*, 16(3):368–373, 2006.
- [3] JSBB. Journal of concepts in structural biology & bioinformatics, 2023. Accessed: 10 June 2026.

Appendix A

Attached Python Code for MSA and Phylogenetic Tree

The following Python code was used to perform BLAST retrieval, MSA, and phylogenetic tree construction (Neighbor-Joining with Bootstrap).

phylogenetic pipeline

June 11, 2026

Calling important files and libraries

```
[1]: from Bio import SeqIO

# To open and read the FASTA file
record = SeqIO.read("NM_000518.5.fasta", "fasta")

print("ID:", record.id)
print("Description:", record.description)
print("Sequence:")

print(record.id)
print(record.seq)
```

ID: NM_000518.5

Description: NM_000518.5 Homo sapiens hemoglobin subunit beta (HBB), mRNA

Sequence:

NM_000518.5

ACATTTGCTTCTGACACAACCTGTGTTCCTAGCAACCTCAAACAGACACCATGGTGCATCTGACTCCTGAGGAGAAGTCT
GCCGTTACTGCCCTGTGGGGCAAGGTGAACGTGGATGAAGTTGGTGGTGAGGCCCTGGGCAGGCTGCTGGTGGTCTACCC
TTGGACCCAGAGGTTCTTTGAGTCCTTTGGGGATCTGTCCACTCCTGATGCTGTTATGGGCAACCCTAAGGTGAAGGCTC
ATGGCAAGAAAGTCTCGGTGCCTTTAGTGATGGCCTGGCTCACCTGGACAACCTCAAGGGCACCTTTGCCACACTGAGT
GAGCTGCACTGTGACAAGCTGCACGTGGATCCTGAGAACTTCAGGCTCCTGGGCAACGTGCTGGTCTGTGTGCTGGCCCA
TCACTTTGGCAAAGAATTCACCCACCAAGTGCAGGCTGCCTATCAGAAAGTGGTGGCTGGTGTGGCTAATGCCCTGGCCC
ACAAGTATCACTAAGCTCGCTTTCTTGCTGTCCAATTTCTATTAAGGTTCTTTGTTCCCTAAGTCCAACCTACTAACT
GGGGGATATTATGAAGGGCCTTGAGCATCTGGATTCTGCCTAATAAAAAACATTTATTTTCATTGCAA

```
[2]: from Bio.Blast import NCBIWWW
from Bio import SeqIO
```

Storing sequence as string and getting first 100 bases

```
[3]: from Bio import SeqIO

# Read the FASTA file
record = SeqIO.read("NM_000518.5.fasta", "fasta")

# To store the sequence as a string
sequence = str(record.seq)
```

```
print("Sequence length:", len(sequence))
print(sequence[:100]) # to see first 100 bases
```

Sequence length: 628
ACATTTGCTTCTGACACAACCTGTGTTCACCTAGCAACCTCAAACAGACACCATGGTGCATCTGACTCCTGAGGAGAAGTCT
GCCGTTACTGCCCTGTGGG

To run nucleotide blastn

```
[17]: from Bio.Blast import NCBIWWW

# To run BLAST (blastn for nucleotide sequences)
result_handle = NCBIWWW.qblast(
    program="blastn",
    database="nt",
    sequence=sequence
)
```

To store the blast result as xml

```
[18]: with open("blast_results.xml", "w") as out_handle:
    out_handle.write(result_handle.read())
```

To print the BLAST result

```
[19]: from Bio.Blast import NCBIXML

with open("blast_results.xml") as result_handle:
    blast_record = NCBIXML.read(result_handle)

for alignment in blast_record.alignments[:5]: # Top 5 hits
    print("Title:", alignment.title)
    print("Length:", alignment.length)

    for hsp in alignment.hsps[:1]: # First HSP
        print("Score:", hsp.score)
        print("E-value:", hsp.expect)
        print("Identities:", hsp.identities)
        print("Percent identity:", round((hsp.identities / hsp.align_length) * 100, 2), "%")
        print("-" * 50)
```

Title: gi|1401724401|ref|NM_000518.5| Homo sapiens hemoglobin subunit beta (HBB), mRNA
Length: 628
Score: 1256.0
E-value: 0.0
Identities: 628
Percent identity: 100.0 %

Title: gi|2468480781|ref|XM_508242.5| PREDICTED: Pan troglodytes hemoglobin subunit beta (HBB), mRNA
Length: 639
Score: 1251.0
E-value: 0.0
Identities: 627
Percent identity: 99.84 %

Title: gi|2694496772|ref|XM_003819029.5| PREDICTED: Pan paniscus hemoglobin subunit beta (LOC100976465), mRNA
Length: 643
Score: 1246.0
E-value: 0.0
Identities: 626
Percent identity: 99.68 %

Title: gi|13937928|gb|BC007075.1| Homo sapiens hemoglobin, beta, mRNA (cDNA clone MGC:14540 IMAGE:4292125), complete cds
Length: 658
Score: 1239.0
E-value: 0.0
Identities: 624
Percent identity: 99.52 %

Title: gi|29436|emb|V00497.1| Human messenger RNA for beta-globin
Length: 626
Score: 1237.0
E-value: 0.0
Identities: 623
Percent identity: 99.52 %

Running BLAST in comparison to one outlier organism
in this case mouse

```
[20]: from Bio.Blast import NCBIWWW
      from Bio import SeqIO

      # Reading FASTA file
      record = SeqIO.read("NM_000518.5.fasta", "fasta")
      sequence = str(record.seq)

      # doing BLAST against an outlier group: mouse
      result_handle = NCBIWWW.qblast(
          program="blastn",
          database="nt",
```

```

sequence=sequence,
entrez_query="Mus musculus[Organism]"
)

# Saving the results as xml file
with open("mouse_outlier_blast.xml", "w") as out_file:
    out_file.write(result_handle.read())

```

printing of Blast result in case of mouse

```

[21]: from Bio.Blast import NCBIXML

with open("mouse_outlier_blast.xml") as result_handle:
    blast_record = NCBIXML.read(result_handle)

for alignment in blast_record.alignments[:5]:
    print("Title:", alignment.title)
    print("Length:", alignment.length)

    hsp = alignment.hsps[0]
    print("Score:", hsp.score)
    print("E-value:", hsp.expect)
    print("Identities:", hsp.identities)
    print("Percent identity:", round((hsp.identities / hsp.align_length) * 100,
↵2), "%")
    print("-" * 50)

```

Title: gi|12846885|dbj|AK011033.1| Mus musculus 13 days embryo liver cDNA, RIKEN full-length enriched library, clone:2510031C09 product:hemoglobin, beta adult major chain, full insert sequence

Length: 627

Score: 585.0

E-value: 4.26913e-147

Identities: 502

Percent identity: 79.3 %

Title: gi|12849365|dbj|AK012551.1| Mus musculus 11 days embryo whole body cDNA, RIKEN full-length enriched library, clone:2700082N11 product:hemoglobin, beta adult major chain, full insert sequence

Length: 646

Score: 584.0

E-value: 4.26913e-147

Identities: 500

Percent identity: 79.37 %

Title: gi|12832344|dbj|AK002394.1| Mus musculus adult male kidney cDNA, RIKEN full-length enriched library, clone:0610009G19 product:hemoglobin, beta adult major chain, full insert sequence >gi|12838093|dbj|AK005496.1| Mus musculus

adult female placenta cDNA, RIKEN full-length enriched library, clone:1600019I13
product:hemoglobin, beta adult major chain, full insert sequence
>gi|74214435|dbj|AK168584.1| Mus musculus 13 days embryo liver cDNA, RIKEN full-
length enriched library, clone:I920037A19 product:hemoglobin, beta adult major
chain, full insert sequence >gi|74223004|dbj|AK168819.1| Mus musculus 13 days
embryo liver cDNA, RIKEN full-length enriched library, clone:I920057K24
product:hemoglobin, beta adult major chain, full insert sequence
Length: 627
Score: 584.0
E-value: 4.26913e-147
Identities: 500
Percent identity: 79.37 %

Title: gi|12846915|dbj|AK011050.1| Mus musculus 13 days embryo liver cDNA, RIKEN
full-length enriched library, clone:2510039C10 product:hemoglobin, beta adult
major chain, full insert sequence
Length: 626
Score: 584.0
E-value: 4.26913e-147
Identities: 500
Percent identity: 79.37 %

Title: gi|74198365|dbj|AK167615.1| Mus musculus 14 days embryo liver cDNA, RIKEN
full-length enriched library, clone:I530025D15 product:hemoglobin, beta adult
major chain, full insert sequence
Length: 627
Score: 584.0
E-value: 4.26913e-147
Identities: 500
Percent identity: 79.37 %

```
[22]: '''
      entrez_query="Gallus gallus[Organism]"    # chicken
      entrez_query="Danio rerio[Organism]"      # zebrafish
      entrez_query="Xenopus laevis[Organism]"   # frog
      '''
```

```
[22]: '\nentrez_query="Gallus gallus[Organism]"    # chicken\nentrez_query="Danio
      rerio[Organism]"      # zebrafish\nentrez_query="Xenopus laevis[Organism]" #
      frog\n'
```

1 Multiple sequence Alignment

collecting the sequences for alignment

Human HBB (NM_000518.5)

Chimpanzee HBB

Bonobo HBB

Mouse HBB (outlier)

```
[23]: sequence = str(record.seq)
      human_sequence = sequence # your NM_000518.5 sequence

      chimp_sequence = "PASTE_CHIMP_SEQUENCE_HERE"
      bonobo_sequence = "PASTE_BONOBO_SEQUENCE_HERE"
      mouse_sequence = "PASTE_MOUSE_SEQUENCE_HERE"
```

Saving them in one fasta file called hbb_sequences.fasta

```
[24]: with open("hbb_sequences.fasta", "w") as f:
      f.write(">Human_HBB\n")
      f.write(human_sequence + "\n")

      f.write(">Chimp_HBB\n")
      f.write(chimp_sequence + "\n")

      f.write(">Bonobo_HBB\n")
      f.write(bonobo_sequence + "\n")

      f.write(">Mouse_HBB\n")
      f.write(mouse_sequence + "\n")
```

```
[25]: from Bio.Blast import NCBIXML

      with open("blast_results.xml") as result_handle:
          blast_record = NCBIXML.read(result_handle)

      for alignment in blast_record.alignments[:3]:
          print(alignment.accession)
```

NM_000518

XM_508242

XM_003819029

```
[26]: with open("mouse_outlier_blast.xml") as result_handle:
      mouse_record = NCBIXML.read(result_handle)

      for alignment in mouse_record.alignments[:3]:
          print(alignment.accession)
```

AK011033

AK012551

AK002394

```
[27]: mouse_accession = mouse_record.alignments[0].accession
      print(mouse_accession)
```

AK011033

```
[28]: accessions = [
      "NM_000518.5",      # Human
      "XM_508242.5",      # Chimpanzee
      "XM_003819029.5",   # Bonobo
      "AK011033.1"        # Mouse outgroup
      ]
```

Downloading actual fasta sequence using the accession numbers

```
[29]: from Bio import Entrez, SeqIO

      Entrez.email = "ramgurung.info@example.com" # replace with your email

      accessions = [
          "NM_000518.5",      # Human
          "XM_508242.5",      # Chimpanzee
          "XM_003819029.5",   # Bonobo
          "AK011033.1"        # Mouse outgroup
      ]

      handle = Entrez.efetch(
          db="nucleotide",
          id=",".join(accessions),
          rettype="fasta",
          retmode="text"
      )

      records = list(SeqIO.parse(handle, "fasta"))

      SeqIO.write(records, "hbb_sequences.fasta", "fasta")

      print("Saved", len(records), "sequences to hbb_sequences.fasta")
```

Saved 4 sequences to hbb_sequences.fasta

checking whether that the FASTA file was created correctly.

```
[30]: from Bio import SeqIO

      records = list(SeqIO.parse("hbb_sequences.fasta", "fasta"))

      for record in records:
          print(record.id)
          print("Length:", len(record.seq))
```

```
print(record.seq[:50])
print("-" * 40)
```

```
NM_000518.5
Length: 628
ACATTGCTTCTGACACAACCTGTGTTCCTAGCAACCTCAAACAGACACC
-----
XM_508242.5
Length: 639
ATCTATTGCTTACATTTGCTTCTGACACAACCTGTGTTCCTAGCAACCTC
-----
XM_003819029.5
Length: 643
AGCCATCTATTGCTTACATTTGCTTCTGACACAACCTGTGTTCCTAGCAA
-----
AK011033.1
Length: 627
GACGTTTGCTTCTGATTCTGTTGTGTTGACTTGCAACCTCAGAAACAGAC
-----
```

Now running MSA

```
[31]: import shutil

print("muscle:", shutil.which("muscle"))
print("clustalo:", shutil.which("clustalo"))
```

```
muscle: None
clustalo: None
```

```
[32]: conda install -c bioconda mafft
```

```
Jupyter detected...
2 channel Terms of Service accepted
Channels:
- bioconda
- defaults
Platform: linux-64
Collecting package metadata (repodata.json): done
Solving environment: done
```

```
==> WARNING: A newer version of conda exists. <==
current version: 26.3.2
latest version: 26.5.2
```

Please update conda by running

```
$ conda update -n base -c defaults conda
```

Package Plan

environment location: /home/ram-gurung/miniconda3

added / updated specs:

- mafft

The following packages will be downloaded:

package	build	
openssl-3.5.7	h1b28b03_0	5.5 MB
Total:		5.5 MB

The following packages will be UPDATED:

openssl 3.5.6-h1b28b03_0 --> 3.5.7-h1b28b03_0

Downloading and Extracting Packages:

Preparing transaction: done

Verifying transaction: done

Executing transaction: done

Note: you may need to restart the kernel to use updated packages.

```
[1]: import subprocess
```

```
subprocess.run([
    "mafft",
    "--auto",
    "hbb_sequences.fasta"
], stdout=open("hbb_aligned.fasta", "w"))
```

outputthat23=16

treein = 0

compacttree = 0

stacksize: 8192 kb

generating a scoring matrix for nucleotide (dist=200) ... done

All-to-all alignment.

tbfast-pair (nuc) Version 7.525

```
alg=L, model=DNA200 (2), 2.00 (6.00), -0.10 (-0.30), noshift, amax=0.0
0 thread(s)
```

```
outputhat23=16
Loading 'hat3.seed' ...
done.
Writing hat3 for iterative refinement
generating a scoring matrix for nucleotide (dist=200) ... done
Gap Penalty = -1.53, +0.00, +0.00
tbtree = 1, compacttree = 0
Constructing a UPGMA tree ...
    0 / 4
done.
```

```
Progressive alignment ...
STEP      3 /3
done.
tbfast (nuc) Version 7.525
alg=A, model=DNA200 (2), 1.53 (4.59), -0.00 (-0.00), noshift, amax=0.0
1 thread(s)
```

```
minimumweight = 0.000010
autosubalignment = 0.000000
nthread = 0
randomseed = 0
blosum 62 / kimura 200
poffset = 0
niter = 16
sueff_global = 0.100000
nadd = 16
Loading 'hat3' ... done.
generating a scoring matrix for nucleotide (dist=200) ... done

    0 / 4
Segment  1/ 1    1- 646
STEP 002-002-1 identical.
Converged.
```

```
done
dvtitr (nuc) Version 7.525
alg=A, model=DNA200 (2), 1.53 (4.59), -0.00 (-0.00), noshift, amax=0.0
0 thread(s)
```

Strategy:
L-INS-i (Probably most accurate, very slow)
Iterative refinement method (<16) with LOCAL pairwise alignment information

If unsure which option to use, try 'mafft --auto input > output'.
For more information, see 'mafft --help', 'mafft --man' and the mafft page.

The default gap scoring scheme has been changed in version 7.110 (2013 Oct).
It tends to insert more gaps into gap-rich regions than previous versions.
To disable this change, add the --leavegappyregion option.

```
[1]: CompletedProcess(args=['mafft', '--auto', 'hbb_sequences.fasta'], returncode=0)
```

```
[2]: import shutil
      print(shutil.which("mafft"))
```

/home/ram-gurung/miniconda3/bin/mafft

```
[4]: import subprocess

      mafft_path = "/home/ram-gurung/miniconda3/bin/mafft"

      with open("hbb_aligned.fasta", "w") as output_file:
          subprocess.run(
              [mafft_path, "--auto", "hbb_sequences.fasta"],
              stdout=output_file,
              check=True
          )

      print("Alignment completed and saved as hbb_aligned.fasta")
```

Alignment completed and saved as hbb_aligned.fasta

outputthat23=16
treein = 0
compacttree = 0
stacksize: 8192 kb
generating a scoring matrix for nucleotide (dist=200) ... done
All-to-all alignment.
tbfast-pair (nuc) Version 7.525
alg=L, model=DNA200 (2), 2.00 (6.00), -0.10 (-0.30), noshift, amax=0.0
0 thread(s)

outputthat23=16
Loading 'hat3.seed' ...
done.
Writing hat3 for iterative refinement
generating a scoring matrix for nucleotide (dist=200) ... done
Gap Penalty = -1.53, +0.00, +0.00
tbtree = 1, compacttree = 0
Constructing a UPGMA tree ...
0 / 4

```

done.

Progressive alignment ...
STEP      3 /3
done.
tbfast (nuc) Version 7.525
alg=A, model=DNA200 (2), 1.53 (4.59), -0.00 (-0.00), noshift, amax=0.0
1 thread(s)

minimumweight = 0.000010
autosubalignment = 0.000000
nthread = 0
randomseed = 0
blosum 62 / kimura 200
poffset = 0
niter = 16
sueff_global = 0.100000
nadd = 16
Loading 'hat3' ... done.
generating a scoring matrix for nucleotide (dist=200) ... done

      0 / 4
Segment  1/ 1    1- 646
STEP 002-002-1  identical.
Converged.

done
dvtdditr (nuc) Version 7.525
alg=A, model=DNA200 (2), 1.53 (4.59), -0.00 (-0.00), noshift, amax=0.0
0 thread(s)

Strategy:
  L-INS-i (Probably most accurate, very slow)
  Iterative refinement method (<16) with LOCAL pairwise alignment information

If unsure which option to use, try 'mafft --auto input > output'.
For more information, see 'mafft --help', 'mafft --man' and the mafft page.

The default gap scoring scheme has been changed in version 7.110 (2013 Oct).
It tends to insert more gaps into gap-rich regions than previous versions.
To disable this change, add the --leavegappyregion option.

```

Viewing alignment

```
[5]: from Bio import AlignIO
```

```
alignment = AlignIO.read("hbb_aligned.fasta", "fasta")

print("Number of sequences:", len(alignment))
print("Alignment length:", alignment.get_alignment_length())
print(alignment)
```

```
Number of sequences: 4
Alignment length: 645
Alignment with 4 rows and 645 columns
-----acatttgcttctgacacaactgtgttcac...caa NM_000518.5
----atctattgcttacatttgcttctgacacaactgtgttcac...caa XM_508242.5
agccatctattgcttacatttgcttctgacacaactgtgttcac...caa XM_003819029.5
-----gacgtttgcttctgattctgttggttgac...c-- AK011033.1
```

viewing alignment w.r.t. conserved regions, gaps, probable variations

step 1. loading fasta file and checking alignment size

```
[6]: from Bio import AlignIO

# Load MSA result
alignment = AlignIO.read("hbb_aligned.fasta", "fasta")

# Basic summary
print("Number of sequences:", len(alignment))
print("Alignment length:", alignment.get_alignment_length())

for record in alignment:
    print(record.id, "length:", len(record.seq))
```

```
Number of sequences: 4
Alignment length: 645
NM_000518.5 length: 645
XM_508242.5 length: 645
XM_003819029.5 length: 645
AK011033.1 length: 645
```

step.2 identifying conserved and variable positions

```
[7]: from collections import Counter

alignment_length = alignment.get_alignment_length()

conserved = 0
variable = 0

for i in range(alignment_length):
    column = alignment[:, i]

    # Ignore gaps when counting
```

```

residues = [x for x in column if x != "-"]

if len(set(residues)) == 1:
    conserved += 1
else:
    variable += 1

print("Conserved positions:", conserved)
print("Variable positions:", variable)
print("Percent conserved:", round(conserved/alignment_length*100, 2), "%")

```

Conserved positions: 510
Variable positions: 135
Percent conserved: 79.07 %

identifying gap positions

```

[8]: gap_positions = []

for i in range(alignment.get_alignment_length()):
    column = alignment[:, i]

    if "-" in column:
        gap_positions.append(i + 1)

print("Number of gap positions:", len(gap_positions))
print("First 20 gap positions:", gap_positions[:20])

```

Number of gap positions: 21
First 20 gap positions: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 55, 56, 516, 539, 644]

Finding highly variable sites

```

[9]: for i in range(alignment.get_alignment_length()):
    column = alignment[:, i]

    residues = set(column)

    if len(residues) > 1:
        print("Position", i+1, ":", column)

```

Position 1 : --a-
Position 2 : --g-
Position 3 : --c-
Position 4 : --c-
Position 5 : -aa-
Position 6 : -tt-
Position 7 : -cc-
Position 8 : -tt-

Position 9 : -aa-
Position 10 : -tt-
Position 11 : -tt-
Position 12 : -gg-
Position 13 : -cc-
Position 14 : -tt-
Position 15 : -ttg
Position 18 : aaag
Position 30 : ccct
Position 31 : aaat
Position 33 : aaat
Position 34 : aaag
Position 35 : ccct
Position 42 : cccg
Position 46 : aaat
Position 55 : ---a
Position 56 : ---g
Position 66 : ccct
Position 76 : tccc
Position 83 : cccg
Position 84 : ccca
Position 87 : aaac
Position 88 : gggt
Position 95 : tttg
Position 100 : ccct
Position 103 : tttc
Position 104 : aaat
Position 108 : cccg
Position 118 : ccca
Position 129 : tttc
Position 130 : gggc
Position 169 : gggt
Position 188 : aaac
Position 192 : ttta
Position 199 : gggt
Position 200 : ttta
Position 201 : cccg
Position 208 : ggga
Position 211 : tttc
Position 214 : ggga
Position 218 : aaat
Position 221 : cccg
Position 223 : tttc
Position 224 : gggt
Position 225 : aaac
Position 230 : ggga
Position 232 : tttc
Position 238 : ccct

Position 241 : ccct
Position 242 : cccg
Position 244 : tttc
Position 247 : ggga
Position 256 : tttc
Position 272 : ccca
Position 274 : ccca
Position 275 : ggga
Position 276 : gggc
Position 285 : ggga
Position 286 : tttc
Position 296 : ggga
Position 297 : ccca
Position 302 : ccct
Position 309 : aaag
Position 330 : cccg
Position 331 : aaac
Position 334 : gggc
Position 343 : gggc
Position 361 : ccct
Position 394 : ccct
Position 395 : ggga
Position 398 : ccca
Position 400 : gggc
Position 403 : cccg
Position 404 : ttta
Position 405 : gggt
Position 414 : cccg
Position 418 : tttc
Position 422 : tttc
Position 424 : ttgt
Position 430 : aaag
Position 433 : aaat
Position 442 : aaac
Position 443 : cccg
Position 445 : aaat
Position 447 : tttc
Position 448 : ggga
Position 459 : aaat
Position 460 : tttc
Position 466 : aaag
Position 478 : ttta
Position 485 : aaag
Position 486 : aaac
Position 496 : ccct
Position 505 : tttc
Position 514 : tttc
Position 516 : ggg-

Position 521 : ccct
Position 522 : tttc
Position 527 : ggga
Position 529 : ccct
Position 530 : cccg
Position 531 : aaat
Position 532 : aaac
Position 534 : ttta
Position 536 : cccg
Position 537 : ttcc
Position 539 : ttt-
Position 540 : tttc
Position 548 : ccca
Position 549 : ccct
Position 550 : ttta
Position 552 : tttg
Position 553 : gggt
Position 554 : tttc
Position 555 : tttc
Position 558 : ccct
Position 559 : ttta
Position 560 : aaag
Position 563 : ttta
Position 564 : ccca
Position 565 : ccca
Position 570 : aaag
Position 571 : ccct
Position 572 : tttc
Position 575 : aaat
Position 576 : ccct
Position 577 : tttg
Position 578 : gggt
Position 584 : ttta
Position 587 : tttg
Position 594 : ggga
Position 595 : gggc
Position 597 : ccct
Position 601 : aaag
Position 608 : ggga
Position 610 : aaac
Position 613 : ccct
Position 615 : ggga
Position 616 : ccct
Position 625 : aaat
Position 626 : aaag
Position 628 : ccct
Position 634 : tttc
Position 636 : tttg

```

Position 641 : tttc
Position 644 : aaa-
Position 645 : aaa-

```

2 Phylogenetic tree making

```

[10]: from Bio import AlignIO

alignment = AlignIO.read("hbb_aligned.fasta", "fasta")

print("Number of sequences:", len(alignment))
print("Alignment length:", alignment.get_alignment_length())

```

```

Number of sequences: 4
Alignment length: 645

```

```

[11]: from Bio.Phylo.TreeConstruction import DistanceCalculator

calculator = DistanceCalculator("identity")

distance_matrix = calculator.get_distance(alignment)

print(distance_matrix)

```

```

NM_000518.5 0.000000
XM_508242.5 0.018605    0.000000
XM_003819029.5 0.026357    0.007752    0.000000
AK011033.1 0.218605    0.232558    0.237209    0.000000
    NM_000518.5 XM_508242.5 XM_003819029.5 AK011033.1

```

```

[12]: from Bio.Phylo.TreeConstruction import DistanceTreeConstructor
from Bio import Phylo

constructor = DistanceTreeConstructor()

tree = constructor.nj(distance_matrix)

Phylo.draw_ascii(tree)

```

```

, NM_000518.5
|
|   _ XM_003819029.5
-|---|
|   | XM_508242.5
|
|----- AK011033.1

```



```
[13]: from Bio import Phylo
import matplotlib.pyplot as plt

# Make figure bigger
plt.figure(figsize=(10, 6))

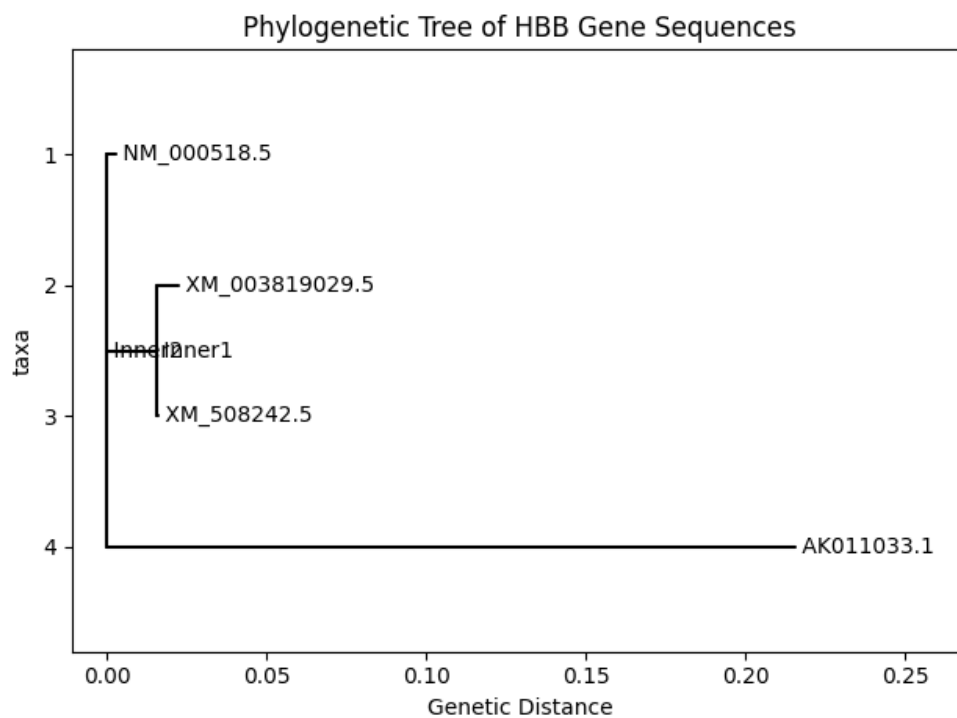
# Draw tree
Phylo.draw(
    tree,
    do_show=False
)

plt.title("Phylogenetic Tree of HBB Gene Sequences")
plt.xlabel("Genetic Distance")
plt.tight_layout()

# Save beautiful tree image
plt.savefig("hbb_phylogenetic_tree.png", dpi=300)

plt.show()
```

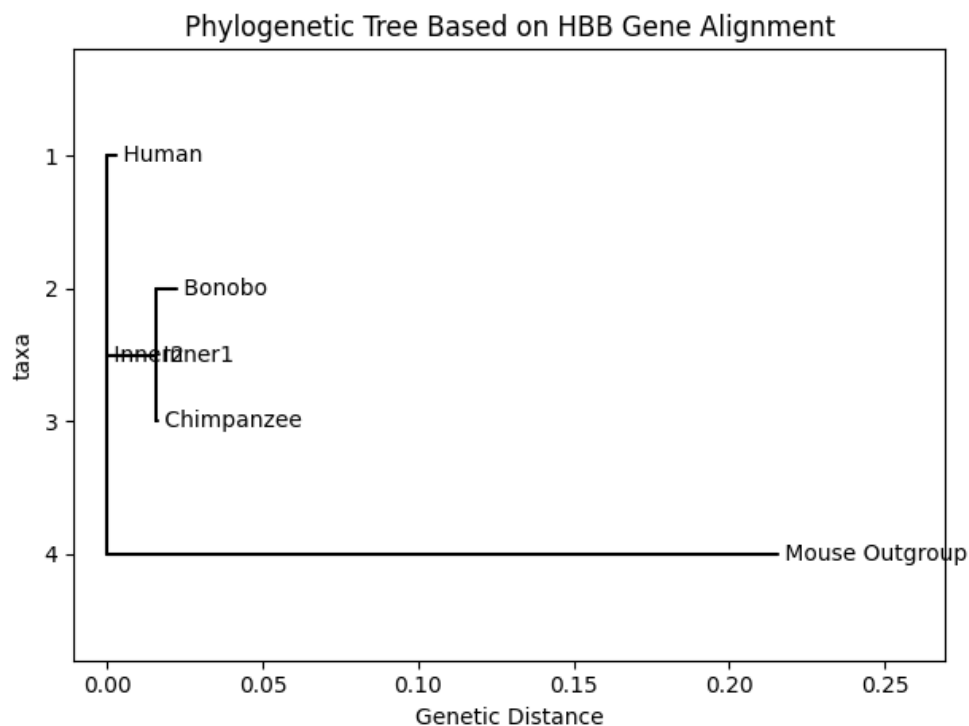
<Figure size 1000x600 with 0 Axes>



```
[14]: # Rename labels before drawing
for clade in tree.find_clades():
    if clade.name == "NM_000518.5":
        clade.name = "Human"
    elif clade.name == "XM_508242.5":
        clade.name = "Chimpanzee"
    elif clade.name == "XM_003819029.5":
        clade.name = "Bonobo"
    elif clade.name == "AK011033.1":
        clade.name = "Mouse Outgroup"

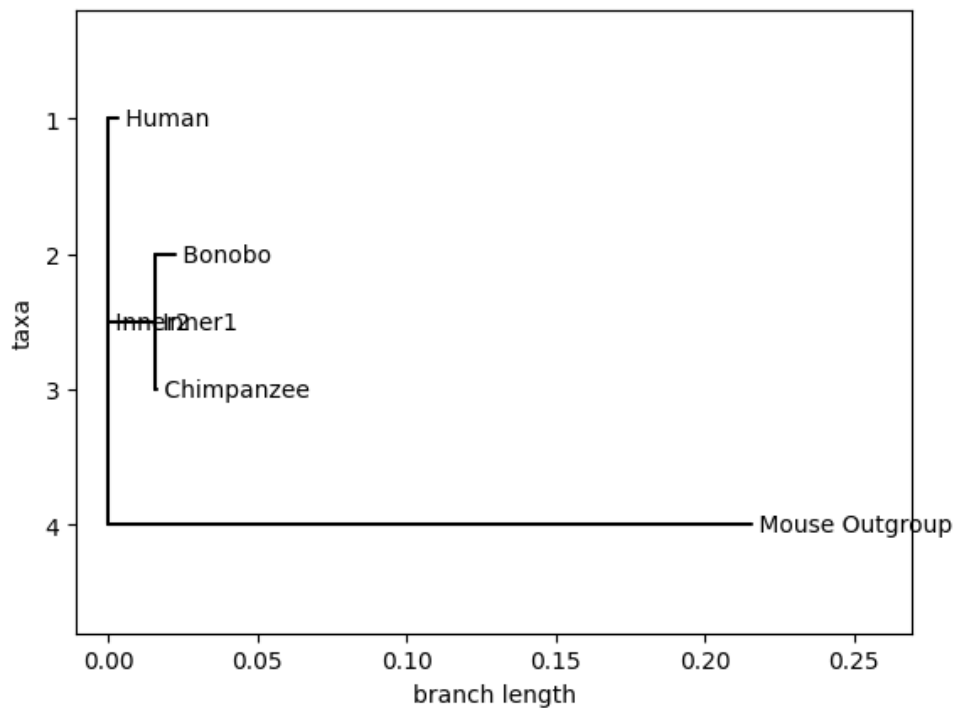
plt.figure(figsize=(10, 6))
Phylo.draw(tree, do_show=False)
plt.title("Phylogenetic Tree Based on HBB Gene Alignment")
plt.xlabel("Genetic Distance")
plt.tight_layout()
plt.savefig("hbb_phylogenetic_tree_labeled.png", dpi=300)
plt.show()
```

<Figure size 1000x600 with 0 Axes>



```
[15]: plt.figure(figsize=(10,6))
Phylo.draw(tree, do_show=False)
plt.savefig("phylogenetic_tree.png", dpi=300)
plt.show()
```

<Figure size 1000x600 with 0 Axes>



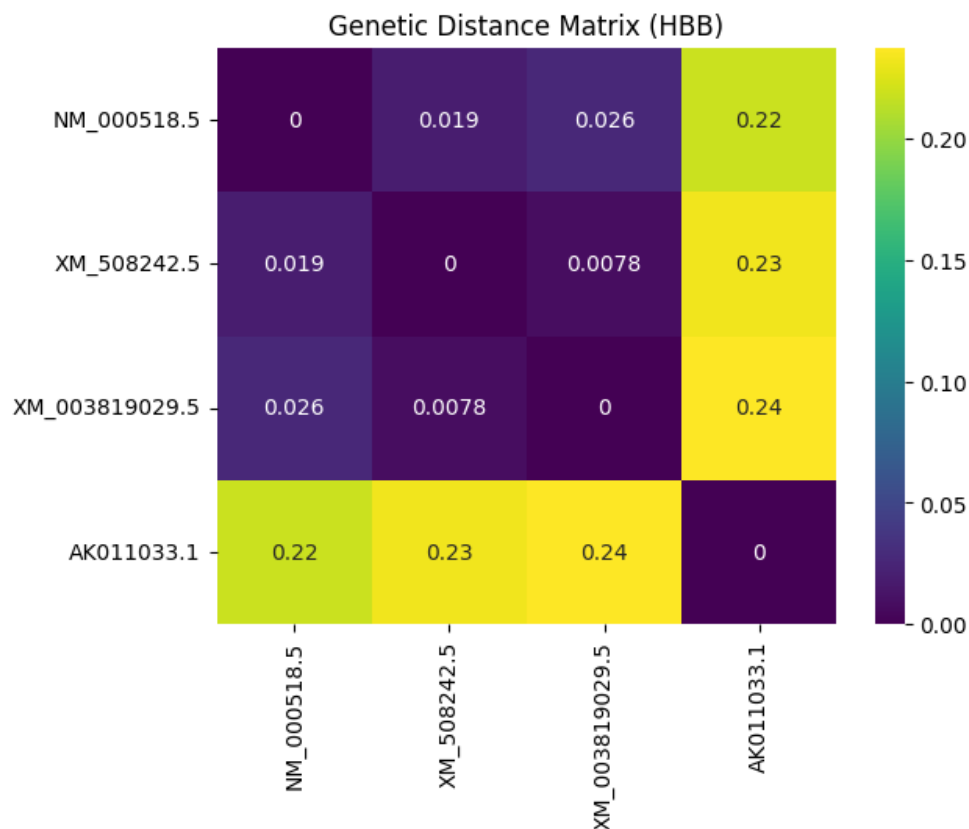
```
[16]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Convert distance matrix to DataFrame
names = distance_matrix.names

df = pd.DataFrame(
    distance_matrix.matrix,
    index=names,
    columns=names[:len(distance_matrix.matrix)]
)
```

```
# Fix lower-triangular matrix to full matrix
for i in range(len(df)):
    for j in range(i):
        df.iloc[j, i] = df.iloc[i, j]

sns.heatmap(df, annot=True, cmap="viridis")
plt.title("Genetic Distance Matrix (HBB)")
plt.show()
```



```
[17]: import sys
      !{sys.executable} -m pip install scipy
```

Collecting scipy

Using cached scipy-1.17.1-cp313-cp313-

manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (62 kB)

Requirement already satisfied: numpy<2.7,>=1.26.4 in /home/ram-gurung/miniconda3/lib/python3.13/site-packages (from scipy) (2.4.6)

Downloading

```
scipy-1.17.1-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (35.2 MB)
```

```
35.2/35.2 MB 15.1 MB/s 0:00:02 eta  
0:00:010:01:01
```

```
Installing collected packages: scipy  
Successfully installed scipy-1.17.1
```

```
[ ]:
```